

0. Keyword Index by Exam Term

Category	Search these terms / go to sections
DBMS & Architecture	DBMS purpose, efficiency, reliability (1); centralised, distributed, cloud, grid, P2P, web (4)
Storage & I/O	HDD, SSD, seek, rotation, cache, buffer (2); RAID, availability, CRC, reliable write/message, WAL, ARIES, checkpoint (24–31)
SQL & Query	SQL, relational algebra, selectivity, cardinality, statistics, enumeration, heuristic, adaptive, rewrite, nested-loop join (5–9)
Indexes & Spatial	index choice, hash, B+ tree, bitmap, R-tree, quadtree, EXPLAIN, fragmentation (10–13)
Transactions	ACID, isolation, flat transaction, savepoint, cursor, nested transaction, parent/child, TP monitor, 2PC (14–16, 32)
Concurrency & Locks	dirty read, lost update, conflict, dependency graph, serialisability, wormhole, S/X, 2PL, isolation degree, deadlock, intention lock, update lock, OCC, snapshot (17–23, 33)
Distributed Consistency	distributed transaction, replication, available copies, CAP, eventual consistency, BASE, PACELC (32–35)
Backup & Warehouse	remote backup, one-safe/two-safe, full/differential/incremental backup, data warehouse, ETL (36–37)

1. DBMS Purpose, Efficiency and Reliability

Key points:

- A DBMS stores an integrated collection of data and provides controlled, efficient and reliable access.
- Efficiency:** response time and throughput; affected by storage, I/O, memory, architecture, data model, indexes and query plans.
- Effectiveness:** concurrent users obtain correct results, not merely fast results.
- Security/reliability:** access control, transactions, recovery, replication and fault tolerance preserve data and service.

Answer structure: State the required property, identify the workload/failure/concurrency assumption, name the mechanism, then explain its benefit and cost. **Worked example:** “The system must serve many users after a node failure” requires availability and fault tolerance; replication helps, but adds coordination and consistency cost.

2. HDD, SSD, Cache and Buffer I/O

$$T_{HDD} = T_{seek} + T_{rotation} + \frac{\text{transfer size}}{\text{bandwidth}}, \quad T_{SSD} = \frac{\text{transfer size}}{\text{bandwidth}}$$

- Seek moves the head to a track; rotation waits for the sector; transfer moves the bytes.
- Large sequential I/O is dominated by transfer time; small random I/O is dominated by seek/rotation.

Worked example: With seek 12 ms, rotation 4 ms, bandwidth 4 MB/s: 8 MB takes $12 + 4 + 2000 = 2016$ ms; 8 KB takes about 18 ms. The small request spends most of its time positioning the disk.

$$EA = HC + (1 - H)M, \quad EA_{buffer} = H_B B_C + (1 - H_B)D$$

- Weight hit and miss paths by probability; compare the final effective time, not hit ratio alone.
- Tutorial example: $0.5C + 0.5(100C) = 50.5C$, while $0.9C + 0.1(400C) = 40.9C$; the larger-cache machine is faster despite slower memory.

Exam trap: Keep units consistent. Convert MB/s to KB/ms before adding millisecond latencies.

3. NoSQL: Key-Value, Document, Column, Graph

Model	Strength and suitable workload	Limitation
Key-value	key → value; direct lookup, carts, sessions, caching; simple and scalable	Weak joins, relationships and rich predicates
Document	JSON-like, nested and varying fields; articles/catalogues and evolving semi-structured records	Cross-document joins and constraints are weaker
Column-based	Reads selected columns efficiently; large analytical scans and aggregation	Less natural for frequent whole-record retrieval/update
Graph	Vertices and edges; multi-hop traversal, recommendations, friend-of-friend and paths	Extra complexity when relationships are not central

- Select from the **dominant access pattern**, not merely the input format.
- NoSQL commonly trades some relational joins, constraints, query flexibility or consistency guarantees for schema flexibility and horizontal scale.

Answer structure: Identify the data structure and main operations; choose the model; explain the matching strength; finish with one lost capability. **Exam trap:** NoSQL means “Not Only SQL”, not “no SQL”.

4. Architecture: Centralised, Distributed, Cloud

- Centralised:** simple administration and consistency; limited scale and one failure domain.
- Distributed:** owned sites share data/work; locality, scale and fault tolerance improve, but global coordination is harder.
- Cloud:** elastic third-party infrastructure and low initial capital cost; privacy, control, vendor dependence and recurring cost matter.
- Grid:** autonomous nodes contribute resources to a task. **P2P:** dynamic decentralised membership. **Web:** globally distributed data with different owners.

Worked example: Startup with one office, little capital and expected rapid growth: cloud is attractive because capacity can scale without building data centres. A large organisation requiring direct ownership/privacy across its own cities may prefer a distributed database. **Answer structure:** Compare scale, ownership, privacy, availability, budget and administration. Borderline answers are acceptable when the trade-off is explicit.

5. SQL, Relational Algebra and Query Pipeline

SQL → parse/translate → relational algebra
→ physical plans → lowest estimated cost → execute

- Selection σ chooses rows; projection π chooses columns; product $\times$ forms all pairs; theta join $\bowtie_{\theta}$ applies a condition; natural join $\Join$ uses common attributes.
- Example: `SELECT name FROM E WHERE salary>80000` becomes $\pi_{name}(\sigma_{\text{salary}>80000}(E))$.
- Logical operators say *what*; heap/index scan and nested-loop/hash/merge join say *how*.
- Equivalent expressions return the same result but may create very different intermediate sizes.

Answer structure: Identify the query result, map clauses to logical operators, propose a useful rewrite, then discuss the physical access path.

6. Selectivity, Cardinality and Statistics

$$|\sigma_{\theta}(R)| \approx RF(\theta)|R|, \quad RF(A = v) \approx \frac{1}{V(A, R)}$$
$$RF(\theta_1 \wedge \theta_2) \approx RF(\theta_1)RF(\theta_2) \quad \text{if independent}$$
$$|R \Join_{A=B} S| \approx \frac{|R||S|}{\max(V(A, R), V(B, S))}$$

- $V(A, R)$: distinct A -values. Keep tuple count n_R , page count b_R and distinct count $V(A, R)$ separate.
- Uniformity and independence are assumptions. Skew and correlated predicates can make estimates wrong.
- Statistics guide plan selection; sampling and histograms improve estimates.

Worked example: After many insertions/deletions, total rows may be unchanged but distributions/statistics may be stale. The optimiser then misestimates selectivity and chooses a formerly good but now poor plan. Refresh/recompile statistics and inspect the new plan.

7. Query Optimisation: Enumeration, Heuristic, Adaptive

- Exhaustive enumeration:** cost many equivalent plans and select the cheapest. Best justified for expensive, complex multi-join queries; optimisation overhead/search space can be large.
- Heuristic:** apply rules such as early selection/projection and restrictive operations first. Fast and suitable for simple queries, but not guaranteed globally optimal.
- Adaptive:** execute part of a cost-based plan, observe actual cardinality and choose/revise the remaining plan. Useful when estimates are uncertain.
- Real systems often use heuristics to prune the search space before cost-based comparison.

Worked example: Tutorial scenarios: a single-table query with two predicates can use heuristics; a five-table join benefits from cost-based enumeration, possibly adaptive re-estimation. A purely heuristic plan has no cost model to adapt in the same sense. **Answer structure:** Compare query complexity, optimisation time, estimation uncertainty and the value of runtime feedback.

8. Query Rewriting, Materialisation and Tuning

- Push selective predicates down: $\sigma_c(R \Join S) \equiv \sigma_c(R) \Join S$ when c uses only R .
- Push projections down to reduce tuple width, but retain join, filter and output attributes.
- Optimise the complete plan; the cheapest isolated operator may create an expensive intermediate result.
- Refresh statistics; add an appropriate index; parameterise/rewrite repeated queries; force a plan only when the optimiser repeatedly fails.
- Store derived/pre-joined data when computation is frequent and source changes are infrequent; pay refresh/storage cost.

Worked example: If R has one million rows but only 1% satisfy $R.status = 'active'$, filtering before the join reduces the candidate rows to about 10,000.

9. Nested-Loop Join Cost: Tuple vs Page

Let r be outer, s inner, n_r outer tuples and b_r, b_s page counts.

Tuple NLJ: $I/O = b_r + n_r b_s, \quad seeks = b_r + n_r$

Page NLJ: $I/O = b_r + b_r b_s, \quad seeks = 2b_r$

- Test both orientations: making the smaller relation outer often reduces repeated inner scans.
- Page NLJ reuses one outer page across an inner scan and is normally much cheaper than tuple NLJ.
- Merge join is attractive for ordered inputs; hash join for equality; neither choice should be justified without predicate/order/size assumptions.

Answer structure: Define n versus b , substitute carefully, evaluate both orientations, and state the cheaper plan. **Exam trap:** Under the course convention, tables currently in main memory do not change the optimiser's page-cost calculation. This is a sample-exam rule.

10. Index Choice: Hash, B+ Tree, Bitmap, R-tree

- **Equality on ID/name:** Hash or B+ tree. Hash gives direct bucket lookup; B+ tree also supports ordered lookup.
- **Range/inequality/order:** B+ tree. Ordered leaves support range scans and sorted traversal.
- **Low-cardinality filters:** Bitmap. Compact bit vectors support fast AND/OR over values such as status/state.
- **Region/distance/nearest neighbour:** R-tree or quadtree. Spatial partitions/boxes prune irrelevant regions.
- **Aggregate over every row:** usually no filtering index. Every record still contributes, so a scan may be cheaper.
- Always give **index type + search key + predicate + reason**.
- Sample exam: income < 70,000 → B+ tree on income; within 5 km → R-tree/quadtree on location; average income → no filtering index.
- A full scan may win for a small table, low-selectivity predicate or query needing most rows.
- Clustered B+ tree helps when adjacent key values qualify; unclustered indexes may cause many random page reads.
- Composite B+ tree (A, B, C): use leftmost prefixes $A, A, B, A, B, C; B$ alone normally cannot seek.
- **Answer pattern:** state access pattern/selectivity, name the index, explain avoided pages/work, then mention write and space cost.
- Equality-only unique lookup favours hash; use B+ tree when range/order/prefix matters. Bitmap suits read-mostly analytics; frequent updates make maintenance unattractive.

Worked example: WHERE salary > 0: likely no filtering index because most rows qualify. WHERE id=?: hash/B+ tree because it is highly selective. **Answer structure:** “Use a B+ tree with search key income because the predicate defines an ordered range; the tree finds the first qualifying key and scans adjacent leaves.”

11. Index Costs, EXPLAIN, Fragmentation, Statistics

- Benefits: fewer page reads for selective lookup; ordered range access; possible join/order/group support; a covering index may avoid base-table reads.
 - Costs: disk and buffer-pool space; every INSERT adds entries; DELETE removes them; indexed-column UPDATE removes/reinserts entries; writes and administration become slower.
 - Splits, merges, sparse pages and fragmentation can arise; unused/duplicate indexes provide cost without benefit.
 - **Why it matters:** An indexed UPDATE may still use the index to locate target rows quickly. It then pays base-page writes, logging/locking and index maintenance; if the indexed key changes, the old entry is deleted and a new entry inserted. **Diagnosing slowdown after insert/delete**
 - **Stale statistics:** distributions changed, so cardinality estimates and the selected plan are wrong. Refresh statistics and compare estimates with actual rows.
 - **Fragmentation/sparse pages:** splits and deletions increased pages or random I/O. Measure it, then reorganise/rebuild when material.
 - The unchanged row count does not imply unchanged distribution, page layout, clustering or cache behaviour.
- ```
CREATE INDEX idx ON R(A); DROP INDEX idx;
SELECT DISTINCT title FROM titles;
SELECT COUNT(title) FROM titles WHERE title='Engineer';
```
- **EXPLAIN:** estimated access method, index, rows, filtering and join order.
  - **EXPLAIN ANALYZE:** executes and adds actual rows, time and loops. Large estimate error suggests stale statistics/skew/correlation.
  - Using **where:** filtering occurs. Using **index:** possibly covering; it does not prove a small scan.

**Answer structure:** Compare the same query/data/cache conditions. Identify the changed operator, first large estimated-versus-actual row divergence, avoided pages/loops/sort, and measured runtime/buffer effect. Do not claim improvement from the index name alone.

12. B+ Tree: Height, Search, Insert/Delete

$$h_{\max} = \lceil \log_{\lceil n/2 \rceil} K \rceil$$

- $n$ : maximum children;  $K$ : search-key values. High fan-out keeps the tree shallow; every root-to-leaf path has equal depth.
- Internal nodes route; linked leaves contain ordered keys/record pointers.
- Equality descends to one leaf. Range finds the first key then follows leaf links.
- Insert: leaf insertion, split on overflow, propagate separator; root split increases height. Delete: borrow/redistribute, otherwise merge and propagate.

**Worked example:**  $K = 10,000,000$ ;  $n = 4 \Rightarrow \lceil \log_2 K \rceil = 24$ ;  $n = 100 \Rightarrow \lceil \log_{50} K \rceil = 5$ . For key “Crick”, compare at each separator, descend left of

Mozart and Einstein, find Crick in the leaf, then follow its record pointer.

13. Hash, Bitmap, Quadtree and R-tree

Hash

- Hash key to bucket; excellent equality performance when distribution is uniform; no useful key order for ranges.

Bitmap

- One bitmap per value, one bit per record. For states  $[VIC, NSW, WA, VIC, NSW]$ :  $VIC = 10010$ ,  $NSW = 01001$ ,  $WA = 00100$ .
- Combine predicates with bitwise operations, e.g.  $VIC \vee NSW = 11011$ .

Quadtree/R-tree

- Quadtree recursively divides space. R-tree groups objects in minimum bounding rectangles.
- Best-first nearest neighbour: prioritise the node/object with smallest minimum distance; stop only when the nearest queued object cannot be beaten.
- Overlapping R-tree boxes may require multiple branches. Tutorial example: point  $i$  lies in BB1 and BB3, so both zero-distance branches must be expanded; object  $G$  is then nearest.

14. Transactions, ACID and Isolation

- **Atomicity:** all actions occur or none. **Consistency:** constraints hold before/after a correct transaction.
- **Isolation:** concurrent execution has the effect of some serial execution, without requiring literal one-at-a-time execution.
- **Durability:** committed effects survive failure.
- Serial execution is correct but wastes parallel CPU/I/O and lets long transactions block everyone; isolation seeks serial-equivalent results with concurrency.

**Distinguish the properties:** atomicity asks whether all effects survive together; isolation asks what concurrent/intermediate effects others may observe; durability asks whether a committed result survives a crash. **Worked example:** A transfer debits  $A$  and credits  $B$ . Atomicity prevents debit without credit; consistency preserves the total-balance invariant; isolation hides the half-completed transfer; durability preserves both updates after commit. **Answer structure:** Define the property, name the anomaly/failure it prevents, then explain how concurrency/recovery mechanisms enforce it.

15. Flat Transactions, Savepoints and Cursor

- **Flat:** one unit; failure can discard all work. Poor for a million-row bonus update because one late error wastes a long transaction.
- **Savepoints:** rollback to an intermediate state; earlier work is retained but becomes durable only at final commit.
- **Chained:** linked committed segments. **Nested:** parent invokes sub-transactions.
- BEGIN WORK; COMMIT WORK; ROLLBACK WORK;
- Tutorial trace rule: restore the exact variable/database state recorded at the named savepoint, then continue from there.
- A singleton SELECT returns one row directly; a cursor points into a multi-row result and fetches rows one at a time.

**Worked example:** If a rollback returns to savepoint 2 where  $AccBalance1 = 100$ , later changes setting it to 0 disappear; a second rollback to savepoint 2 restores 100 again.

16. Nested Transactions, Parent/Child and TP Monitor

Nested rules

- Child commit is provisional: durable only when every ancestor commits.
- If a subtransaction rolls back, all its descendants roll back.
- Child changes become visible to its parent after child commit; parent objects are visible to children.

**Worked example:** If  $A, B$  and  $C$  commit but PARENT aborts, all roll back. If  $A, B$  and PARENT commit while  $C$  aborts,  $A/B$ /PARENT are durable and  $C$  is rolled back. **TP monitor**

- Integrates clients, services and heterogeneous DBMSs; coordinates communication, context, restart and global transaction work.
- One process for every service is a bottleneck and concentrated failure point; distribution improves performance and fault containment.

17. Dirty Read, Lost Update and Conflict Types

- **Dirty read:** read an uncommitted value that may be rolled back.
- **Unrepeatable read:** two reads of the same object see an intervening committed write.
- **Lost update:** one write overwrites a result computed from an earlier value by another transaction.

Order on object  $O$  Dependency

|                             |                        |
|-----------------------------|------------------------|
| $W_i(O) \rightarrow W_j(O)$ | write–write            |
| $W_i(O) \rightarrow R_j(O)$ | write–read             |
| $R_i(O) \rightarrow W_j(O)$ | read–write             |
| $R_i(O) \rightarrow R_j(O)$ | no conflict dependency |

**Worked example:** With  $B = 50$ ,  $T_1 : \times 2, \times 2$  and  $T_2 : +10$ : serial outcomes are 210 or 240. Interleaving after the first multiply can produce 220, which matches neither serial order and violates isolation.

18. Dependency Graph, Serialisability, Wormhole

If  $T_i$  performs a conflicting action on  $O$  before  $T_j$ :

$$(T_i, O, T_j) \in DEP(H)$$

1. Group actions by object and preserve time order.
2. Ignore read–read; add directed edges for W–W, W–R and R–W.
3. A directed cycle means no equivalent serial history. A DAG topological order gives a serial order; more than one topological order may be valid.  
 $Before(T) = \{U \mid U \rightsquigarrow T\}$ ,  $After(T) = \{U \mid T \rightsquigarrow U\}$

$$Before(T) \cap After(T) \neq \emptyset \iff T \text{ is in a cycle/wormhole}$$

**Worked example:** Tutorial cycle  $T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_1$  is not serialisable. Reversing the  $O_1$  edge gives  $T_3 \rightarrow T_2 \rightarrow T_1$ , a valid serial order (independent  $T_5$  may be placed consistently). **Worked example:** If  $W_1(A)$  precedes  $R_2(A)$ , add  $T_1 \rightarrow T_2$ . If  $W_2(B)$  precedes  $R_1(B)$ , add  $T_2 \rightarrow T_1$ . The two-edge wormhole proves non-serialisability. **Answer structure:** List edges with their object/conflict, draw/check the graph, state cycle or topological order, then conclude isolated/not isolated.

19. S/X Locks, Well-Formedness and 2PL

| Current \ Request | S | X |
|-------------------|---|---|
| Free              | Y | Y |
| S                 | Y | N |
| X                 | N | N |

- READ needs S or stronger; WRITE needs X.
- **Well formed:** every READ/WRITE/UNLOCK is covered by an earlier appropriate lock.
- **Two phase:** growing phase acquires locks; after the first unlock, shrinking begins and no new lock may be acquired.
- If all transactions are well formed and two phase, every legal lock history is isolated. 2PL ensures serialisability but may deadlock.

**Three separate checks:** legal means currently granted locks are compatible; well formed means operations hold the required lock; 2PL means no acquisition/upgrade after the first release/downgrade. A history can pass one and fail another. **Why it matters:** Each transaction has a lock point at its final acquisition. Ordering transactions by lock point is consistent with their conflicts, preventing a dependency cycle. **Variants:** strict 2PL holds X locks to commit/abort and prevents cascading aborts; rigorous 2PL holds both S and X locks. Neither eliminates deadlock. **Worked example:** In a numbered trace, mark the first UNLOCK; every later LOCK line violates 2PL. Per-operation lock/unlock alone is insufficient because releasing between related operations can still produce a non-serial result. **Answer structure:** “The history is legal, but  $T_2$  requests  $X(B)$  after releasing  $S(A)$ , so  $T_2$  violates 2PL and the 2PL serialisability theorem cannot be applied.”

## 20. Isolation Degrees 0–3

| Degree | Guarantee                                                | Required locking                                                |
|--------|----------------------------------------------------------|-----------------------------------------------------------------|
| 0      | Weak conflict protection                                 | Writes well formed                                              |
| 1      | No lost updates                                          | X locks well formed and 2PL; reads may be unlocked              |
| 2      | No lost updates/dirty reads; unrepeatable reads possible | Reads/writes well formed; X locks 2PL; S may release after read |
| 3      | Repeatable reads/full conflict protection                | Complete well-formed 2PL for S and X                            |

|      |                                                     |
|------|-----------------------------------------------------|
| $D1$ | $W \rightarrow W$                                   |
| $D2$ | $W \rightarrow W, W \rightarrow R$                  |
| $D3$ | $W \rightarrow W, W \rightarrow R, R \rightarrow W$ |

**Answer structure:** Find the weakest operation: an unlocked READ prevents Degree 2/3; an X-lock sequence that is not two phase prevents Degree 1+.

## 21. Deadlock, Livelock, Spin Locks, Semaphores

- Deadlock is a cycle in a wait-for/resource-dependency graph; each transaction holds something the next needs.
- Prevent with global resource order or preallocation; detect cycles and roll back a victim; timeout is practical for distributed deadlocks.
- Immediate repeated rollback can cause livelock. Risk rises with concurrency, locks per transaction, hold time, hotspots and write conflicts.
- OS locks sleep but pay context-switch cost; spin locks use atomic test-and-set/CAS and suit short low-contention waits; semaphores queue waiters.
- FIFO handoff can create a convoy; waking all waiters reduces serial handoff but causes competition.

**Worked example:** Tutorial approximation with  $n$  other transactions,  $r$  exclusive locks each and  $R$  records gives lifetime wait probability  $\approx nr^2/(2R)$  and pair-deadlock probability growing roughly with  $n^2r^4/(4R^2)$ . Use this only under the stated simplifying assumptions.

## 22. Granular Locks: IS, IX, SIX, U, X

| Request \ Current | IS | IX | S | SIX | U | X |
|-------------------|----|----|---|-----|---|---|
| IS                | Y  | Y  | Y | Y   | Y | N |
| IX                | Y  | Y  | N | N   | N | N |
| S                 | Y  | N  | Y | N   | N | N |
| SIX               | Y  | N  | N | Y   | N | N |
| U                 | Y  | N  | Y | N   | Y | N |
| X                 | N  | N  | N | N   | N | Y |

- Rows are requests; columns are current modes. The matrix is directional for U/S in this course.
- IS intends finer S; IX intends finer S/X; SIX is S here plus IX below; U reads with intent to update.
- Acquire root-to-leaf, release leaf-to-root. S/IS on a non-root needs a suitable parent; X/U/SIX/IX needs IX or stronger on every ancestor.

## Request procedure

- Process stated arrival order and record only locks already **granted**.
- Verify the required intention mode on every ancestor.
- Compare the request with every granted target lock using the matrix.
- Grant only if all checks pass; otherwise wait. A waiting request is not a current lock.
- State queue policy: strict FIFO may block a later compatible request behind an earlier incompatible one.

**Answer structure:** For every request state ancestor status, current granted modes, compatibility result, grant/wait decision and resulting lock table. A compatible leaf lock cannot override a root conflict.

**Worked example:** Sample pattern: T1's IS and T2's SIX can co-exist at root, so T1/T2 run; later T3's S conflicts with existing SIX and waits. If T3 arrives before T2, T3's S can instead make T2 wait. Arrival order changes the runnable set.

## 23. Update Locks, Escalation, OCC, Snapshot

- U lock admits one intended updater while allowing compatible earlier readers under the course matrix; it reduces S-to-X conversion deadlock.
- Lock escalation replaces many fine locks with a coarse S/X lock: less management overhead, larger conflict range and less concurrency.
- Pessimistic/2PL:** conflicts expected; prevent them before access; cost is locks, waits and deadlocks.
- Optimistic:** conflicts rare; read/compute freely, validate near commit; cost is wasted work/retry when conflicts are common.
- Snapshot isolation:** versioned consistent reads and first-writer-commits; high read concurrency but write skew can violate cross-row constraints.
- Timestamp ordering:** globally consistent order; distributed form carries a coordinator-assigned timestamp to every server.

**Worked example:** Two transactions read  $A = B = 100$ , then independently set  $A = -100$  and  $B = -100$ . No direct write conflict occurs, yet  $A + B \geq 0$  fails: snapshot isolation is not automatically serialisable.

## 24. Probability, Availability, RAID 0/1/5/6

$$P(A \cap B) = P(A)P(B)$$

$$P(A \cup B) \approx P(A) + P(B) \quad \text{for small independent probabilities}$$

$$\text{Availability} = \frac{MTTF}{MTTF + MTTR}, \quad P(\text{unavailable}) \approx \frac{MTTR}{MTTF}$$

If failure requires  $k$  of  $n$  disks simultaneously:

$$P_{\text{fail}} \approx \binom{n}{k} p^k, \quad MTTF_{\text{system}} \approx \frac{M^k}{\binom{n}{k}}$$

| RAID  | Organisation   | Tolerates | Capacity   | Course example      |
|-------|----------------|-----------|------------|---------------------|
| 0     | Block striping | 0         | $nS$       | 2 disks: $M/2$      |
| 1     | Mirroring      | $n - 1$   | $S$        | 2: $M^2$ , 3: $M^3$ |
| 3/4/5 | Single parity  | 1         | $(n - 1)S$ | 3 disks: $M^2/3$    |
| 6     | Dual parity    | 2         | $(n - 2)S$ | 5 disks: $M^3/10$   |

- RAID 0: throughput/capacity, no tolerance. RAID 1: best simple redundancy, lowest utilisation. RAID 4: parity-disk write bottleneck. RAID 5 distributes parity. RAID 6 pays extra parity/write cost for two failures.
- Tutorial utilisation: RAID 0 100%, RAID 1 with two disks 50%, RAID 3 with three disks 2/3.
- Derive reliability by finding the minimum simultaneous failures that lose data, counting such combinations, multiplying by  $p^k$ , then inverting the failure rate under the course approximation.

**Worked example:** Six 2-TB disks provide 12 TB raw. RAID 1 provides 6 TB usable; RAID 5 provides  $5 \times 2 = 10$  TB. Capacity and reliability are separate calculations. **Worked example:** RAID 0 loses data after any one of  $n$  disks fails, so  $P_{\text{loss}} \approx np$ . A mirrored pair requires both failures,  $p^2$ . Single parity tolerates one failure but loses data for any pair,  $\binom{n}{2}p^2$ . **Exam trap:** Tutorial text has a typo for RAID 0; doubled failure probability means MTTF halves, so use  $M/2$ . State the independence/small- $p$  assumption.

## 25. Fault Tolerance: Failvote, Failfast, CRC

## Failvote vs failfast

- Failvote requires a majority of the original modules: stronger confidence, lower availability as failures accumulate.
- Failfast excludes unavailable modules and takes a majority among working modules: higher availability, but one remaining module cannot be checked.

## Reliable block write

- Duplex write stores two sequential copies with version numbers; choose the newest consistent copy.
- Logged write stores the second representation in a log and is efficient for small changes.
- CRC detects storage/transmission corruption and burst errors.

**Answer structure:** Separate *correctness of an accepted result* from *availability of the service*.

## 26. Reliable Message Passing: Out, Ack, In

$$Out = \#sent, \quad Ack = \#acknowledged, \quad In = \#received$$

When B sends message  $k$  to A:

- B persistently records  $Out_k$  and sets  $Out = k$ .
  - B sends; A persistently records  $In_k$  and sets  $In = k$ .
  - A acknowledges  $k$ ; B persistently sets  $Ack = k$ .
- Worked example:** Initial  $A$  ( $Out = 3, Ack = 3, In = 6$ ),  $B$  ( $Out = 6, Ack = 6, In = 3$ ). B sends message 7 and receives the acknowledgement:

$$A : (3, 3, 7), \quad B : (7, 7, 3)$$

**Answer structure:** First identify sender and receiver. Only sender changes Out/Ack; only receiver changes In. Preserve stable-storage update order so retransmission/recovery is possible.

## 27. Buffer Pool, STEAL/NO-FORCE, WAL

- fix(page)** loads/pins; a fixed page cannot be evicted. **unfix** releases one user's pin. Dirty means memory is newer than disk.
- STEAL allows uncommitted dirty pages to reach disk  $\rightarrow$  recovery needs UNDO/old values.
- NO-FORCE allows commit before every dirty page reaches disk  $\rightarrow$  recovery needs REDO/new values.
- ARIES supports efficient STEAL + NO-FORCE.

## Write-ahead logging

- Before a data page is written, its update log must be durable:

$$pageLSN \leq flushedLSN.$$

- Before commit returns, the log through the commit record must be durable.

**Why it matters:** Rule 1 ensures an on-disk uncommitted update can be undone; Rule 2 ensures a committed update absent from data pages can be redone.

## 28. Log Records: LSN, pageLSN, CLR

| Field                                                                                                                                                                                                                                                                                                                                                                                                                             | Meaning                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| LSN                                                                                                                                                                                                                                                                                                                                                                                                                               | Increasing log-record identifier                             |
| pageLSN                                                                                                                                                                                                                                                                                                                                                                                                                           | Latest update/CLR applied to a page                          |
| flushedLSN                                                                                                                                                                                                                                                                                                                                                                                                                        | Largest durable LSN                                          |
| prevLSN/lastLSN                                                                                                                                                                                                                                                                                                                                                                                                                   | Previous/latest log for a transaction                        |
| recLSN                                                                                                                                                                                                                                                                                                                                                                                                                            | Earliest potentially unpropagated update that dirtied a page |
| undoNextLSN                                                                                                                                                                                                                                                                                                                                                                                                                       | CLR pointer to the next action requiring undo                |
| <ul style="list-style-type: none"> <li>UPDATE: log XID/page/old/new/prevLSN; update lastLSN/pageLSN; first dirtying adds DPT entry with recLSN.</li> <li>COMMIT: log commit, flush through commit LSN, return, later write END.</li> <li>ABORT: log abort, follow prevLSN backwards, undo updates and write CLRs, then END.</li> <li>CLR is redo-only: it records an undo already performed and is never undone again.</li> </ul> |                                                              |



29. ARIES: Analysis, Redo, Undo

Analysis → Redo (repeat history) → Undo losers

**Analysis**

- Start from checkpoint via master record; initialise transaction table and DPT from END\_CHECKPOINT.
- Scan forward: update status/lastLSN; first UPDATE/CLR for a missing page adds DPT recLSN. END removes a transaction.
- Committed/ended work is winner work; active/aborting transactions remaining are losers.

**Redo**

- Start at smallest DPT recLSN. For each UPDATE/CLR, skip if page absent from DPT,  $recLSN > LSN$ , or disk  $pageLSN \geq LSN$ ; otherwise reapply and set pageLSN.
- Redo repeats crash-time history, including loser updates/CLRs; it is not “redo committed only”.

**Undo**

- Initialise *ToUndo* with loser lastLSNs; repeatedly take the largest.
- Ordinary update: restore old value, write CLR, follow prevLSN. CLR: follow undoNextLSN. End of chain: write END.

**Why it matters:** ARIES first restores the exact physical state at crash, then undoes losers once in a controlled order. A CLR records completed undo and is redo-only, so a second crash does not repeat that undo incorrectly. **Interpret the structures:** the transaction table tracks status/log-chain end; DPT recLSN is the earliest update that may be missing from disk for a dirty page; pageLSN shows the latest update already present on that disk page.

30. ARIES Worked Pattern: DPT, TT, ToUndo

**Worked example:** Tutorial log:

```
10 T1 : P1 : Y → Z
15 T2 : P3 : U → V
25 CP : TT = [T1 : 10, T2 : 15], DPT = [P1 : 10, P3 : 15]
30 T1 : P2 : W → X
35 T1 : COMMIT
40 T2 : P1 : Z → T
45 T2 : ABORT
50 T2 : CLR P1 = Z, undoNext = 15
```

- Analysis reconstructs  $P1 : 10, P3 : 15, P2 : 30$ ; T1 is committing/winner, T2 aborting/loser with lastLSN 50.
- Redo starts at 10 and conditionally reapplies 10,15,30,40 and CLR 50 according to DPT/pageLSN.
- Undo starts  $ToUndo = \{50\}$ ; CLR 50 points to 15; undo LSN 15 by restoring  $P3 = U$ , write a CLR, then END.

**Answer structure:** Present the final Analysis transaction table/DPT; a Redo row for each candidate showing DPT, recLSN and pageLSN tests; and a descending ToUndo trace showing each CLR, undoNextLSN and next queue state.

31. Checkpoint, Shadow Paging, Recovery Cost

**Checkpoint trade-off**

- Frequent checkpoints reduce the log/history considered after a crash and shorten recovery.
- They add normal-time log entries, disk I/O and transaction overhead. Choose based on recovery urgency versus throughput.

**Shadow paging**

- Keep immutable shadow and current page tables; update pages by copy-on-write.
- Commit flushes modified pages/current table, then atomically redirects the fixed shadow pointer.
- Advantage: no update log and simple crash recovery. Costs: page-table copying, garbage collection, fragmentation, high commit flushing and weak concurrency support.

32. Distributed Transactions and 2PC

- Atomic commit and global serial order are different: 2PC gives all-commit/all-abort; locking/timestamps/optimistic validation give consistent conflict order.
- Phase 1:** coordinator sends PREPARE; participants force prepare state and vote YES/NO.
- Phase 2:** all YES →record/send GLOBAL COMMIT; any NO/timeout →GLOBAL ABORT; participants record decision, act locally and release locks.
- A YES vote promises that the participant has durable prepare state and

can commit later; it therefore cannot unilaterally abort while uncertain.

**Worked example:**

- P1/P2 YES, P3 NO →abort all.
- Participant crashes before voting →timeout and abort.
- Coordinator crashes after all YES →prepared participants cannot decide independently; request the decision and may block until recovery.

**Why it matters:** Blocking preserves atomicity: committing locally could conflict with another participant’s abort. The cost is retained locks/resources and reduced availability while the durable global decision is unavailable. **Answer structure:** State the durable state already reached, who can still decide, the timeout/recovery action, and why locks/blocking remain.

33. Distributed Concurrency, Replication, Copies

- Locking must preserve the same T-before-U conflict order at every relevant server.
- Timestamp ordering uses a globally unique coordinator timestamp carried with each object access.
- Optimistic distributed control validates at each server during 2PC phase 1 using a globally consistent validation order.
- Available Copies reads one available replica and writes all available replicas, but changing availability sets can hide conflicts and break one-copy serialisability.
- Final conservative rule: before commit, verify that the contacted failed/available set did not change; otherwise abort, even if the particular execution might have been harmless.

**Why local correctness is insufficient:** each server may have a serialisable local schedule but choose a different order for the same two global transactions. If site *A* orders  $T \rightarrow U$  and site *B* orders  $U \rightarrow T$ , the combined dependency graph has a cycle. **Worked example:** Under Available Copies,  $T_1$  may read/write replica *A* while *B* is considered failed. If membership changes and  $T_2$  later uses stale *B* without learning  $T_1$ ’s update, both transactions can appear locally valid but violate one-copy behaviour. **Answer structure:** Explain the desired one-logical-copy behaviour, show how sites could miss one another, then state the validation/abort rule.

34. CAP: Consistency, Availability, Partition

- Consistency:** one current logical value. **Availability:** every request to a non-failing node receives a response. **Partition tolerance:** service continues despite network separation.
- During a partition, strict consistency and availability cannot both be guaranteed.
- Strong consistency requires waiting, coordination or rejecting operations; this raises latency and lowers availability.
- Eventual consistency allows temporary stale reads but replicas converge if updates stop.

**Worked example:** Sample exam: many geographically distributed servers, unreliable links and millions of users. Partitions are realistic; serving users with low latency/high availability is important, so eventual consistency is suitable if temporary staleness is acceptable. **Answer-ready detail**

- CP rejects/delays some partitioned requests to avoid conflicting or stale results.
- AP continues serving but permits temporary divergence; versions, timestamps, quorums, CRDTs or application rules reconcile replicas later.
- State the invariant: banking/payment often cannot accept divergence; feeds, caches or carts may tolerate it.

**Worked example:** Two partitioned cart replicas accept different additions. Availability is preserved and carts temporarily differ; set-union merge can converge later if duplicate/removal semantics are defined. **Answer structure:** Identify partition risk; explain the C/A conflict; connect the choice to user-visible behaviour; state convergence/conflict resolution and its cost.

35. Eventual, Causal, BASE and PACELC

- Strong: every read sees latest. Eventual: convergence later. Causal: preserve cause-before-effect.
- Read-your-writes: see own latest update. Session: guarantee within one session. Monotonic read: never go backwards. Monotonic write: one process’s writes keep issue order.

BASE = Basically Available + Soft State + Eventual Consistency

If partition: A or C; Else: Latency or C

- PA/EL: available under partition, low latency normally. PC/EC: consistency in both cases.
- PA/EC: available under partition, consistency normally. PC/EL: consistent under partition, low latency normally.

- CAP describes the forced choice during partition; PACELC additionally describes latency-versus-consistency during normal operation.
- NoSQL/partitioning link: scalable NoSQL systems often shard data and use BASE-style relaxed consistency to gain availability, latency and partition tolerance.

**Worked example:** Shopping cart/product display may favour availability; checkout/payment needs stronger consistency. Airline booking can tolerate more staleness with many seats than near sell-out. Apply guarantees per business function, not universally. **Answer structure:** Name the weakest model preserving the application invariant; state the anomaly it permits, the availability/latency benefit, and why the anomaly is acceptable or repairable.

36. Remote Backup: One-Safe, Two-Safe

- Heartbeats/multiple links distinguish primary failure from link failure; bad detection can create split brain.
- Backup takeover redoes completed work and removes incomplete work. Periodic redo/checkpointing reduces takeover delay; a hot spare continually applies redo.
- One-safe:** primary log only; low latency, possible acknowledged-data loss before replication.
- Two-very-safe:** primary and backup durable before commit; strongest durability, but site/link failure blocks commit.
- Two-safe:** two-very-safe normally, primary-only during backup outage; better availability but weaker degraded-mode durability.

**Failure-analysis pattern**

- Identify the last log record known durable at primary and backup.
- Decide whether the client already received commit acknowledgement.
- On takeover, redo durable committed work and undo/remove incomplete work.
- Prevent both sites accepting writes simultaneously using a quorum, lease, fencing token or authoritative membership decision.

**Worked example:** If one-safe acknowledges after only the primary log is forced, immediate primary loss before shipping can lose an acknowledged transaction. Two-very-safe prevents that loss but cannot acknowledge while the backup/link is unavailable. **Answer structure:** Compare durability, availability, latency and behaviour during link/site failure.

37. Backup: Full/Differential, Warehouse, ETL

**Backup**

- Full versus partial depends on change scope; differential stores changes since the last full backup.
- Incremental stores changes since the previous backup: smallest daily backup, but restore may need the full backup plus a long chain. Differential grows over time but restore needs only the full plus latest differential.
- Higher frequency reduces possible data loss (RPO) but adds I/O/storage; restore design determines recovery time (RTO).
- Historical copies enable point-in-time recovery.

**Data warehouse**

- Integrates historical data under a unified schema for analytics and moves decision support away from OLTP.
- Source-driven pushes updates; destination-driven pulls. Exact 2PC synchronisation is usually too costly, so some staleness is accepted.
- Requires schema integration, cleansing, deduplication and time dimensions; stored aggregates can replace detail when totals answer the workload.
- OLTP favours current detailed records, short updates and concurrency; a warehouse favours historical snapshots, scans, aggregation and stable analytical schemas.

**Worked example:** A nightly differential plus weekly full backup limits the restore set to two backups, while log archiving between them supports point-in-time recovery. The schedule must still satisfy the required RPO and tested restore time. **Answer structure:** Name the objective, choose the mechanism/frequency, and state freshness, recovery, storage and operational trade-offs. For warehousing, also explain why asynchronous ETL and accepted staleness protect OLTP throughput.